Atividade prática: autenticação e área administrativa

Desafio

Você receberá uma branch inicial deste repositório **sem as alterações de autenticação, login, cadastro e proteção administrativa descritas neste enunciado**. Implemente o fluxo completo usando os recursos do Laravel e os padrões já presentes no projeto.

O objetivo não é apenas criar telas: as rotas, a sessão e o middleware de autorização devem funcionar em conjunto.

Trabalho: ate duas pessoas
Entrega: 14/11/2026

Objetivos de aprendizagem

- Implementar autenticação baseada em sessão.
- Distinguir autenticação de autorização.
- Proteger rotas com middleware.
- Criar formulários Blade independentes do layout administrativo.
- Validar entradas e apresentar erros ao usuário.
- Demonstrar os fluxos HTTP de sucesso e de acesso negado.
- Evitar que um cadastro público conceda privilégios administrativos.

Requisitos funcionais

1. Login

Implemente uma página de login com email e senha. A tela deve:

- Ser acessível por `GET /login`.
- Ser independente de `layouts.default`: exibir somente o formulário, sem cabeçalho, sidebar ou conteúdo do painel.
- Usar o estilo já adotado nas telas do projeto e carregar os assets front-end existentes.
- Enviar as credenciais por `POST /login`, com proteção CSRF.
- Exibir erros de validação e uma mensagem genérica para credenciais inválidas.
- Preservar o email digitado após uma falha, mas nunca repopular a senha.
- Ter um link para a tela de cadastro.

Ao autenticar:

- Regere o identificador da sessão.
- Usuários comuns devem ser encaminhados à página inicial (`/`).
- Usuários com perfil administrativo devem ser encaminhados ao painel (`/admin/users`), respeitando um destino interno solicitado anteriormente quando aplicável.

- O login deve ter limitação de tentativas. Configure o limite adotado no projeto e explique como ele funciona.

2. Logout

Implemente `POST /logout` para usuários autenticados. O encerramento deve:

- Invalidar a sessão atual.
- Regenerar o token CSRF.
- Redirecionar à tela de login.

Adicione no menu do usuário em `resources/views/parts/header.blade.php` um botão **Sair**. O botão deve enviar um formulário `POST` com CSRF; não implemente logout por link `GET`.

3. Middleware administrativo

Crie um middleware que proteja todas as rotas sob `/admin/*`:

- Visitante não autenticado: redirecionar para `/login`.
- Usuário autenticado sem perfil administrativo: responder com `403`.
- Usuário cujo perfil tem `type = admin`: permitir que a requisição prossiga.

Autenticação, por si só, não deve conceder acesso administrativo. Não crie um perfil `admin` durante o cadastro público.

4. Rotas

Organize as rotas para garantir estes comportamentos:

Método e caminho	**Acesso**	**Resultado esperado**
`GET /`	Autenticado	Exibe a página inicial existente.
`GET /login`	Público	Exibe o formulário de login independente.
`POST /login`	Público, com limite de tentativas	Valida credenciais e autentica.
`POST /logout`	Autenticado	Encerra a sessão.
`GET /register`	Público	Exibe o formulário de cadastro independente.
`POST /store`	Público	Valida e cria uma conta comum.
`GET /admin/*` e demais métodos administrativos atuais	Perfil admin	Executa as funcionalidades já existentes do painel.

Preserve os caminhos e nomes de rotas existentes que as views já utilizam. Aplique o middleware ao conjunto administrativo, evitando deixar uma rota administrativa fora da proteção.

5. Cadastro de usuários

Adapte `resources/views/user/create.blade.php` para ser uma página independente, sem `layouts.default`, contendo somente o formulário de cadastro.

O cadastro deve:

- Solicitar nome, email e senha.
- Usar `POST` para a rota de criação e incluir CSRF.
- Validar os campos conforme as regras já usadas pela aplicação, incluindo email único e tamanho mínimo de senha.
- Mostrar erros de validação e preservar os valores não sensíveis.
- Incluir um link de retorno ao login.
- Criar usuários comuns, nunca administradores por padrão.

Após o cadastro público, encaminhe o visitante para o login com uma confirmação. Se um administrador autenticado usar o formulário para incluir um usuário pelo painel, mantenha o retorno à lista administrativa e a mensagem apropriada.

6. Perfil e schema

Antes de alterar modelos ou migrations, confira como o perfil do usuário está armazenado:

- A relação entre `User` e `Profile` deve permitir ao middleware consultar o tipo do perfil.
- O nome da tabela configurado no model deve corresponder ao schema real da branch inicial.
- Confira como o campo de endereço evoluiu nas migrations.
- Não altere migrations já aplicadas ou compartilhadas sem avaliar o impacto; se for necessário corrigir um schema existente, escolha uma estratégia de migração compatível com o estado do banco.

Arquivos a investigar

Use estes caminhos como pontos de partida; a solução pode exigir alterações adicionais dentro da estrutura existente:

- `routes/web.php`
- `app/Http/Controllers/UserController.php`
- `app/Http/Controllers/` para o fluxo de autenticação
- `app/Http/Middleware/` para autorização administrativa
- `app/Models/User.php` e `app/Models/Profile.php`
- `resources/views/auth/` para a tela de login
- `resources/views/user/create.blade.php`
- `resources/views/parts/header.blade.php`
- `database/migrations/`

Cenários para demonstração

Prepare contas de demonstração em ambiente local para demonstrar:

- Visitante tentando abrir `/` e `/admin/users`.
- Cadastro de uma conta comum.
- Login e logout dessa conta.
- Tentativa de acesso administrativo por essa conta, com resposta 403.
- Login de um administrador preparado para a atividade e acesso ao painel.

O usuário administrativo deve ser provisionado de maneira controlada em ambiente local, por exemplo via seeder de desenvolvimento. **Não** crie um formulário público que aceite `type = admin`, nem use uma senha de produção nos exemplos.

Critérios de avaliação

Critério	**Peso sugerido**
Regras de autenticação e autorização implementadas corretamente	30%
Rotas protegidas e preservadas sem brechas	25%
Formulários, validação, mensagens e navegação	25%
Organização, clareza e demonstração do fluxo	20%

Entrega

Entregue:

1. Faça fork do repositorio https://github.com/chsguitar1/aula-alfa-laravel
2. As alterações implementadas na branch no seu repositório e abertura de PR para o repositório principal.
3. Uma demonstração dos cenários descritos, captura de tela.

Perguntas para reflexão

- Por que um usuário autenticado ainda pode receber 403?
- Qual seria o risco de confiar apenas em um botão escondido na interface?
- Por que logout deve ser uma requisição `POST` protegida por CSRF?
- Como o limite de tentativas de login ajuda e quais são suas limitações?
- Como provisionar o primeiro administrador sem expor essa capacidade ao público?

Protótipo: Telas de exemplo





